

HONR2890 Programming Intelligent Robots

Assignment 4: Robot Puppy

Prof. Christopher Crick

1 Instructions

Write a robot controller that allows a Turtlebot to approach a bright yellow ball, slow down to a stop while looking right at it, and then dash toward it to give it a kick. You will create a “robot_puppy” node, which will use a finite state machine, two PID controllers and the ball_detector node you created for the last assignment. I will not be providing skeleton code this time, but you should be able to organize the code much like the prison break assignment.

The robot_puppy node will subscribe to the topic published by the ball detector, providing the ball’s measured bearing and distance. In addition to a robot object, the code should also define a PID object (you can use the code provided in lecture). Create two PIDs, one for bearing and one for distance. The goal bearing is “centered” (ie $x = 125$), while the goal distance is 1.0 meters. You will need separate K_p , K_d and K_i constants for each PID controller.

You will have one subscriber, to handle BallLocation messages from your ball_detector node. The handler assigned to this topic should store the bearing and distance to the ball in instance variables, so that they can be accessed from your state machine loop. If the bearing and/or distance are invalid, this must be communicated somehow.

The robot’s states should be the following:

- **Search:** The robot should slowly rotate in place. If the bearing and distance instance variables contain valid values, transition to the **Approach** state.
- **Approach:** The robot should use its two PID controllers, one on each of the bearing and range, to generate control values for the linear.x and angular.z fields of the Twist message. If either value is invalid, transition back to the **Search** state. If both errors are small enough (exactly how small is up to you), transition to the **Kick** state.
- **Kick:** Drive forward at maximum speed (0.3 m/s) for just enough time (a little over 3 seconds, presumably) to make contact with the ball and send it flying. Afterwards, transition to the **Search** state.

Determining the appropriate values for the PID constants will take some experimentation. Start with just K_p . Figure out the most extreme error you would be able to detect, and set the constant to make that a high but achievable velocity for the robot – about 0.3 m/sec in the linear x dimension and 1.9 rad/sec in the angular z. Test, and adjust from there. If you think that gives you too small a response close to the goal, you can also try increasing the gain (the size of K_p), but thresholding the output. That is, if the computed output is greater than the maximum desired output, set the

output to the maximum desired instead of what the PID is telling you. If the performance of a pure P controller is insufficient, then start messing with K_d and K_i . Use the K_d constant to dampen oscillation and use the K_i constant to overcome bias induced by friction and gear lash.